

DEVOPS MASTER COURSE

Volume 9D

Advanced Ansible Engineering and Enterprise Automation

Chapters 31-40

Roles, collections, execution environments, AWX/controller, Windows, networks, testing, scale, event-driven automation, and enterprise operations

2026 Edition

PACKAGE Roles and collections	EXECUTE Portable environments	OPERATE Controller and evidence
---	---	---

About Volume 9D

Part D moves from individual playbooks to reusable and governed automation products. It develops role and collection contracts, portable execution environments, controller-based operations, Windows and network automation, test engineering, performance control, event-driven response, and enterprise operating practices. The goal is not merely to run more playbooks; it is to make automation reproducible, reviewable, supportable, and safe across teams and platforms.

Labs use disposable systems, containers, simulated APIs, or vendor sandboxes. Production controller, Windows, and network exercises require approved environments and must never be aimed at employer or public infrastructure without authorization. Every lab produces evidence of dependency versions, target scope, pre-change state, validation, idempotency, recovery, and cleanup.

Core principle: enterprise Ansible is a software supply chain and an operating system for change. Content, dependencies, credentials, inventories, execution images, approvals, logs, and recovery procedures must be versioned and governed together.

Volume 9 Structure and Progress

Part	Primary Coverage	Status
9A	Terraform foundations, state, modules, imports, and team workflow	Complete
9B	Advanced Terraform engineering, testing, policy, Stacks, and enterprise operations	Complete
9C	Ansible foundations, inventory, playbooks, templates, handlers, and secrets	Complete
9D	Advanced Ansible roles, collections, execution environments, controller, Windows, networks, testing, and events	This book
9E	Integrated Terraform-Ansible pipelines, GitOps, security, compliance, and observability	Next
9F	Enterprise projects, incidents, certifications, interviews, and integrated capstone	Planned

Part D Chapter Map

Chapter	Focus	Engineering Outcome
31	Roles, argument specifications, dependencies, and reusable contracts	Package configuration logic as a stable automation product.
32	Collections, Galaxy metadata, plugins, semantic versioning, and publishing	Distribute supported content with controlled dependencies.

Chapter	Focus	Engineering Outcome
33	Execution environments, Ansible Builder, Navigator, and Runner	Run identical automation dependencies across local, CI, and controller contexts.
34	AWX and Automation Controller architecture, RBAC, credentials, projects, and inventories	Operate centralized automation with separation of duties.
35	Job templates, surveys, workflows, schedules, notifications, and API integration	Design governed self-service and orchestrated automation services.
36	Windows automation with WinRM, PSRP, SSH, PowerShell, and DSC	Manage Windows safely with platform-aware modules and transports.
37	Network automation, persistent connections, resource modules, backups, and validation	Automate network state without treating devices like Linux hosts.
38	ansible-lint, Molecule, pytest-ansible, CI, and release qualification	Test content behavior, compatibility, idempotency, and failure handling.
39	Strategies, forks, serial, async, throttling, performance, and large-fleet safety	Scale execution while controlling blast radius and dependency pressure.
40	Event-Driven Ansible, rulebooks, observability, governance, and enterprise capstone	Integrate event detection, decision logic, controlled action, and evidence.

Advanced Lab Safety and Evidence Standard

- Use pinned ansible-core, collection, Python, and system-library versions. Retain the dependency files and execution-environment image digest used for each run.
- Treat Galaxy content, container base images, and controller projects as software-supply-chain inputs. Verify source, signatures or attestations where available, support status, and license obligations.
- Keep credentials in approved controller credential types or external secret systems. Never place passwords, private keys, tokens, or Vault password files in repositories or execution images.
- Use organization, team, inventory, project, template, credential, and approval boundaries to enforce separation of duties. A user who can edit code should not automatically be able to execute it against production.
- Windows and network labs must use platform-specific modules and safe connection plugins. Do not use generic shell commands where a supported desired-state module exists.
- Performance tuning begins only after correctness. Increase forks or parallelism gradually while measuring controller resources, target load, API rate limits, connection persistence, and failure amplification.
- For event-driven automation, design suppression, correlation, deduplication, authorization, dry-run or approval paths, circuit breakers, and incident evidence before enabling automatic remediation.

Chapter 31: Roles, Argument Specifications, Dependencies, and Reusable Contracts

Roles turn long playbooks into reusable automation components, but directory structure alone does not create a reliable product. Production roles require explicit inputs, validation, dependency control, idempotency, safe defaults, documented side effects, test scenarios, and versioned compatibility.

Learning Objectives

- Design a role with a narrow responsibility and stable interface.
- Use defaults, vars, handlers, templates, files, and metadata appropriately.
- Define role argument specifications and validate inputs early.
- Manage role dependencies, inclusion semantics, and release compatibility.

Core Concepts

Role boundary: A role should own one coherent capability such as web server configuration, account baseline, or application deployment. Broad “common” roles often become hidden monoliths.

Defaults and vars: defaults/main.yml offers low-precedence configurable defaults. vars/main.yml has high precedence and should be reserved for internal constants that consumers should not override.

Argument specification: meta/argument_specs.yml documents and validates role entry-point options, types, required fields, choices, nested structures, and descriptions.

Static versus dynamic reuse: import_role is static and resolved during parsing; include_role is dynamic and evaluated during execution. This affects tags, conditions, variable visibility, and error timing.

Dependencies: Role metadata can declare dependencies, but hidden execution chains reduce clarity. Explicit orchestration is often easier to review and test.

Role contract: The contract includes supported platforms, required collections, variables, emitted facts, handlers, files changed, services affected, rollback behavior, and compatibility policy.

Contract Area	Good Practice	Failure Risk
Inputs	Typed argument spec with validation	Runtime failure after partial change
Defaults	Safe and environment-neutral	Surprising production behavior
Handlers	Unique names and listen topics	Name collision or unintended restart
Dependencies	Explicit and versioned	Hidden ordering and upgrade breakage
Outputs	Documented facts and status	Consumers depend on internals

Reference Configuration or Command Pattern

```
# roles/web_service/meta/argument_specs.yml
argument_specs:
  main:
    short_description: Configure a managed web service
    options:
      web_service_name:
        type: str
        required: true
      web_service_port:
        type: int
        required: true
      web_service_state:
        type: str
        choices: [started, stopped]
        default: started

# playbooks/site.yml
- name: Configure application tier
  hosts: web
  become: true
  roles:
    - role: web_service
      web_service_name: example
      web_service_port: 8080
```

Production Checklist

- Each role has one named owner and a documented support lifecycle.
- Every external variable is declared in an argument specification or documented compatibility interface.
- Defaults are safe for a disposable environment and do not silently enable destructive behavior.
- Handlers use unique names or listen topics and are tested for unchanged and changed runs.
- Role releases include changelog, supported ansible-core range, collection dependencies, and migration notes.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Role validation fails before tasks	Argument type, required option, or choice mismatch	Correct caller data or role contract; do not bypass validation	Role starts only with valid structured input
Role variable cannot be overridden	Value placed in vars/main.yml or higher-precedence scope	Move consumer-configurable value to defaults or explicit parameter	Caller override works predictably
Handler collision	Two roles use the same global handler name	Use qualified unique names or listen topics	Only intended handler executes
Dynamic include skips expected tags	include_role tag semantics differ from import_role	Apply tags deliberately to include and internal tasks; inspect --list-tags	Selected run executes intended role content

Hands-on Lab 31: Engineer and Test a Contract-Driven Role

Prerequisites

- Volume 9C project or a new isolated Ansible repository.
- A disposable Linux target or container.
- ansible-core and ansible-lint installed.

Implementation

1. Create a web_service role using ansible-galaxy role init.
2. Define defaults, templates, handlers, metadata, README, and meta/argument_specs.yml.
3. Add structured options for service name, port, package list, and enabled state.
4. Implement preflight assertions, package installation, deterministic configuration, and a minimal reload handler.
5. Run with valid inputs, invalid types, an invalid port, and omitted required data.
6. Run twice to prove idempotency and record role side effects and rollback instructions.

Validation Evidence

- Valid invocation passes syntax, lint, and role argument validation.
- Invalid input fails before configuration change.
- First run converges the target; second run reports changed=0.
- README documents inputs, outputs, platforms, dependencies, side effects, tests, and recovery.

Cleanup

- Restore the original service state or delete the disposable target.
- Remove generated credentials and temporary test artifacts.
- Retain the sanitized role, test evidence, and changelog.

Interview Questions and Model Answers

1. Why use role argument specifications?

They create a machine-readable interface that validates inputs early and improves documentation and discoverability.

2. When should a value live in role defaults?

When consumers are expected to configure it and a safe, broadly applicable default exists.

3. What is the difference between import_role and include_role?

Import is static and expanded at parse time; include is dynamic and evaluated during execution, affecting conditions, tags, and variable behavior.

4. Why avoid large common roles?

They combine unrelated ownership and lifecycle, create hidden dependencies, and make testing, releases, and targeted reuse difficult.

Chapter Summary

- A production role is a versioned automation product, not only a directory layout.
- Argument specifications make interfaces testable.
- Defaults, dependencies, handlers, and side effects are part of the compatibility contract.
- Idempotency and recovery must be proven at role level.

Chapter 32: Collections, Galaxy Metadata, Plugins, Semantic Versioning, and Publishing

Collections are the primary distribution format for Ansible content. They can contain roles, modules, plugins, playbooks, documentation, and tests under a namespace. Enterprise collection engineering requires provenance, dependency locks, semantic releases, compatibility testing, documentation, and a controlled publication channel.

Learning Objectives

- Explain collection structure, namespaces, FQCNs, and runtime metadata.
- Create and build a collection with roles, plugins, tests, and documentation.
- Manage collection dependencies and compatibility ranges.
- Publish and consume signed or trusted artifacts through an approved registry.

Core Concepts

Namespace and name: A collection is addressed as namespace.collection. Content uses Fully Qualified Collection Names such as organization.platform.web_service.

galaxy.yml: Defines namespace, name, version, authors, repository, license, tags, dependencies, and build metadata used by ansible-galaxy collection build.

runtime metadata: meta/runtime.yml can declare minimum ansible-core compatibility, plugin routing, redirects, and deprecations.

Content types: Collections can package roles, modules, module_utils, inventory, lookup, filter, callback, connection and other plugins, plus playbooks and docs.

Semantic versioning: Breaking interface or behavior changes require a major release; additive compatible features use minor releases; fixes use patch releases.

Artifact trust: A collection tarball should be built in CI, scanned, checksummed, signed where supported by the distribution channel, stored immutably, and promoted rather than rebuilt per environment.

Artifact	Purpose	Control
galaxy.yml	Package identity and dependencies	Review version and dependency ranges
meta/runtime.yml	Core compatibility and redirects	Porting and deprecation tests
MANIFEST.json	Built artifact file manifest	Reproducible CI build
requirements.yml	Consumer dependency set	Pin or constrain approved versions
FQCN	Unambiguous content identity	Use in production content

Reference Configuration or Command Pattern

```
# galaxy.yml
namespace: acme
name: operations
version: 1.2.0
readme: README.md
license:
  - Apache-2.0
repository: https://example.invalid/acme/operations
dependencies:
  ansible.posix: ">=1.5.0,<3.0.0"

# Build and inspect
ansible-galaxy collection build --force
ansible-galaxy collection install acme-operations-1.2.0.tar.gz -p ./collections
ansible-galaxy collection list -p ./collections

# Consumer requirements.yml
collections:
  - name: acme.operations
    version: ">=1.2.0,<2.0.0"
```

Production Checklist

- Collection namespace and ownership are approved and stable.
- All production tasks use FQCNs to make content provenance explicit.
- Dependencies have reviewed ranges and are tested at minimum and maximum supported versions.
- Builds are produced by CI from a signed tag or protected commit.
- Release notes identify breaking changes, deprecations, security fixes, and required migration steps.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Collection installs but content is not found	Wrong path, namespace, name, or FQCN	Inspect ansible-galaxy collection list and configured COLLECTIONS_PATHS	FQCN resolves from intended artifact
Upgrade breaks playbooks	Unbounded dependency or incompatible major release	Pin or constrain versions; run compatibility matrix	Approved combinations pass tests
Build contains unintended files	Broad build include or missing ignore patterns	Inspect artifact manifest; configure build_ignore	Tarball contains only reviewed content
Redirect or deprecation fails	Incorrect meta/runtime.yml route or removed plugin	Test old FQCN and deprecation path before release	Old reference redirects or fails with planned message

Hands-on Lab 32: Build and Release an Internal Collection

Prerequisites

- A Git repository and Python virtual environment.
- ansible-core, ansible-lint, and ansible-galaxy.
- The role from Chapter 31.

Implementation

1. Create acme.operations with ansible-galaxy collection init.
2. Move the role into the collection and update all calls to its FQCN.
3. Add one simple filter plugin or lookup plugin with documentation and tests.
4. Define galaxy.yml, meta/runtime.yml, changelog fragments or release notes, and requirements files.
5. Build the artifact in a clean directory and inspect the tarball manifest.
6. Install the tarball into a project-local collections path and run the role from the installed artifact.
7. Create a patch release with a backward-compatible fix and prove upgrade compatibility.

Validation Evidence

- The collection builds without untracked or secret files.
- Installed FQCNs resolve from the project-local path.
- Minimum supported ansible-core and dependency versions pass tests.
- Release notes and checksum identify the exact immutable artifact.

Cleanup

- Remove local install paths and tarballs not retained as evidence.
- Confirm repository scans show no credentials or build caches.
- Retain the sanitized source, release artifacts, checksums, and compatibility report.

Interview Questions and Model Answers

1. Why are FQCNs important?

They identify the collection that owns a module or plugin, avoid naming collisions, and improve documentation links and provenance.

2. What belongs in a collection instead of a role?

Collections package multiple content types and shared plugins, establish a namespace, manage dependencies, and support versioned distribution.

3. How should dependency versions be specified?

Use tested constraints that exclude incompatible majors while allowing approved compatible fixes; validate the complete dependency graph in CI.

4. Why promote an artifact instead of rebuilding it?

Promotion preserves the exact tested bytes and provenance across environments, avoiding environment-specific rebuild drift.

Chapter Summary

- Collections are the unit of supported Ansible content distribution.
- FQCNs expose provenance.
- Semantic releases and compatibility matrices protect consumers.
- Build once, verify, and promote immutable artifacts.

Chapter 33: Execution Environments, Ansible Builder, Navigator, and Runner

Python packages, system libraries, collections, and ansible-core versions frequently cause local-versus-CI-versus-controller differences. Execution Environments package the control node as a container image so the same dependency set can run on a workstation, in CI, through AWX, or through Automation Controller.

Learning Objectives

- Explain the contents and lifecycle of an Execution Environment.
- Build a version 3 execution-environment definition with pinned dependencies.
- Use ansible-navigator and ansible-runner to execute and inspect automation.
- Secure, scan, sign, promote, and troubleshoot EE images.

Core Concepts

Execution Environment: A container image intended to act as an Ansible control node, normally containing ansible-core, ansible-runner, Python, collections, Python packages, and system packages.

Ansible Builder: Consumes execution-environment.yml and dependency files to generate a build context and container image.

Ansible Navigator: Runs playbooks in an EE and can inspect image content, collections, documentation, configuration, and execution output.

Ansible Runner: Provides a stable interface and artifact model for invoking Ansible from services, CI, AWX, and custom applications.

Image contract: The contract includes base image digest, Python and ansible-core versions, collections, system packages, non-root behavior, certificates, locale, and supported architectures.

Supply-chain controls: Use approved base images, pinned versions, image scanning, SBOM, signatures or attestations, immutable tags/digests, registry access control, and periodic rebuilds for security fixes.

Layer	Example File	Evidence
Collections	requirements.yml	Installed versions and checksums
Python	requirements.txt	Resolved package list
System	bindep.txt	Installed RPM packages
Definition	execution-environment.yml	Builder version and base digest
Image	Registry digest	SBOM, scan, signature, promotion

Reference Configuration or Command Pattern

```
# execution-environment.yml
version: 3
images:
  base_image:
    name: registry.fedoraproject.org/fedora:43

dependencies:
  ansible_core:
    package_pip: ansible-core==2.20.*
  ansible_runner:
    package_pip: ansible-runner
  galaxy: requirements.yml
  python: requirements.txt
  system: bindep.txt

options:
  package_manager_path: /usr/bin/dnf

# Build and run
ansible-builder build --tag acme-ansible-ee:1.0.0
ansible-navigator images --mode stdout
ansible-navigator run playbooks/site.yml -i inventory --execution-environment-image acme-ansible-ee:1.0.0 --mode
stdout --pull-policy missing
```

Production Checklist

- The base image is approved and referenced by immutable digest for release builds.
- Python, collection, and system dependencies are explicit and tested.
- The image runs as a non-root or dynamically assigned user where the platform supports it.
- Secrets, private keys, Vault passwords, and inventory data are never baked into the image.
- The release record includes builder version, build commit, SBOM, scan, signature, digest, and supported controller version.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Module works locally but not in EE	Missing Python or system dependency in image	Inspect collection requirements and navigator image details; update definition	Same play passes locally through the EE and in CI
Permission denied inside EE	Read-only mounts, dynamic UID, home directory, or file ownership	Use supported mount permissions and runtime paths; avoid root assumptions	EE runs with target platform user model
Collection version differs	Unpinned requirements or registry resolution changed	Pin tested versions and rebuild from protected lock inputs	Image inventory shows expected versions
Controller cannot pull image	Registry credentials, network, TLS trust, or architecture mismatch	Validate registry credential, CA, route, manifest, and digest	Controller launches a minimal job with exact image digest

Hands-on Lab 33: Build, Inspect, and Promote a Custom Execution Environment

Prerequisites

- Podman or Docker and sufficient disk space.
- ansible-builder and ansible-navigator.
- A small collection or role to execute.

Implementation

1. Create execution-environment.yml version 3 plus requirements.yml, requirements.txt, and bindep.txt.
2. Pin ansible-core and the internal collection; include only required dependencies.
3. Build the image and inspect the generated Containerfile and dependency resolution files.
4. Run ansible-navigator images and verify ansible-core, collections, Python, user, and system packages.
5. Execute syntax, check, apply, and idempotency runs inside the EE.
6. Generate or record an SBOM and vulnerability scan; tag the immutable release and record its digest.
7. Simulate promotion by using the same digest from a second project directory.

Validation Evidence

- The exact image passes local and project tests.
- No secret or target-specific inventory exists in image layers.
- The digest, SBOM, scan result, and dependency inventory are recorded.
- A second environment produces the same behavior using the same image.

Cleanup

- Remove unneeded local image layers and build contexts.
- Delete temporary registry credentials and mounted test secrets.
- Retain the sanitized EE definition, test report, SBOM summary, and digest.

Interview Questions and Model Answers

1. What problem do Execution Environments solve?

They package the control-node runtime and dependencies so automation runs consistently across developer, CI, and controller contexts.

2. What should never be included in an EE?

Environment credentials, private keys, Vault passwords, production inventory, target secrets, or mutable environment-specific data.

3. Why record the image digest?

Tags can move; the digest identifies the exact tested image bytes used for a run.

4. How does Ansible Runner relate to AWX?

Runner provides the execution and artifact interface that AWX and other systems use to assemble inputs and invoke Ansible jobs.

Chapter Summary

- Execution Environments make the control node reproducible.
- Dependencies and base images are part of the automation contract.
- Use the same immutable image in local, CI, and controller execution.
- Treat EE images as signed, scanned software artifacts.

Chapter 34: AWX and Automation Controller Architecture, RBAC, Credentials, Projects, and Inventories

Centralized automation provides API-driven execution, audit history, credentials, scheduling, and delegated access, but it also creates a high-impact control plane. Secure operation requires architecture, identity, tenancy, secret handling, project synchronization, capacity planning, backup, and clear separation of duties.

Learning Objectives

- Describe AWX and Automation Controller components and execution flow.
- Design organizations, teams, users, roles, credentials, projects, inventories, and execution environments.
- Apply least privilege and separation of duties to automation resources.
- Plan controller availability, capacity, backup, upgrade, and incident recovery.

Core Concepts

AWX and Controller: AWX is the upstream web UI, REST API, and task engine; Automation Controller is the supported enterprise component in Red Hat Ansible Automation Platform.

Organization boundary: Organizations group users, teams, projects, inventories, credentials, and templates. They are a tenancy and delegation boundary, not a substitute for infrastructure isolation.

Project: A project synchronizes automation content from source control. Production jobs should use protected branches, reviewed commits, and deterministic dependency inputs.

Credential: A controller credential injects authentication material at runtime according to its credential type. Access and use permissions must be separated from secret visibility.

Inventory source: Static or synchronized dynamic inventories define target scope. Sync status, cache age, filters, and failures must be visible before execution.

Instance capacity: Execution is scheduled across controller or execution nodes. Capacity settings, receptor connectivity, job isolation, image pulls, storage, and database health affect reliability.

Resource	Owner	Critical Control
Organization	Platform governance	Delegated administrators and boundaries
Project	Automation engineering	Protected SCM and sync status
Inventory	Service owner	Source, filters, and target review
Credential	Security team	Use access, rotation, external secret source
Template	Operations owner	Prompt limits, approvals, and evidence

Reference Configuration or Command Pattern

```
# Conceptual AWX/controller resource model
Organization: Platform Operations
Teams:
- Automation Developers
- Production Operators
- Security Auditors
Projects:
- infrastructure-automation (protected main branch)
Inventories:
- nonproduction
- production-readonly
- production-managed
Execution Environments:
- acme-ansible-ee@sha256:...
Credentials:
- scm-readonly
- prod-ssh-use-only
Templates:
- Validate Configuration
- Deploy Canary
- Production Rollout
```

Production Checklist

- Human identity uses centralized authentication, MFA, and lifecycle management.
- Source-control edit rights, controller administration, credential administration, and production launch rights are separated.
- Credentials are injected at runtime, rotated, scoped, and preferably sourced from an external secret manager.
- Projects sync protected commits and jobs record the exact SCM revision and EE digest.
- Controller database, configuration, secret keys, execution artifacts, and custom certificates have tested backup and restore procedures.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Project sync succeeds but job uses old content	Update-on-launch disabled, stale cache, or wrong branch	Inspect project revision and template settings; synchronize intentionally	Job records expected protected commit
User can launch with powerful credential	Overbroad resource role or team inheritance	Review object roles and remove unnecessary use permission	Access test confirms least privilege
Jobs remain pending	Insufficient capacity, unavailable execution node, or receptor issue	Inspect instance groups, capacity, node health, and queue constraints	Job schedules on intended node
Inventory differs from source	Failed sync, stale cache, filter change, or duplicate source	Inspect inventory update job and resolved hosts before launch	Host set matches authoritative source

Hands-on Lab 34: Design a Least-Privilege AWX or Controller Tenant

Prerequisites

- AWX lab instance, Automation Controller sandbox, or a documented simulation.
- A Git repository and custom EE digest.
- Test identities for developer, operator, and auditor roles.

Implementation

1. Create an organization, three teams, a project, two inventories, one EE, and test credentials.
2. Assign developers project-update rights without production launch or credential administration.
3. Assign operators launch rights to approved templates; assign auditors read-only access to jobs and activity history.
4. Configure a project sync from a protected branch and record the revision.
5. Launch a nonproduction validation job and inspect events, artifacts, target set, credential use, and EE digest.
6. Document backup, upgrade, outage, capacity, and credential-compromise procedures.

Validation Evidence

- Each test identity can perform only its assigned duties.
- The job record identifies revision, inventory, template, credential reference, and EE.
- No user can reveal credential secret values through the UI or API.
- The operating runbook covers controller, database, registry, SCM, and secret dependencies.

Cleanup

- Remove lab credentials and test users or disable them.
- Delete temporary inventories and projects if the instance is shared.
- Retain sanitized RBAC matrix, architecture, and job evidence.

Interview Questions and Model Answers

1. What is AWX?

The upstream project that provides a web interface, REST API, and task engine on top of Ansible and serves as an upstream for Automation Controller.

2. Why separate project edit and template launch permissions?

It prevents one person from changing automation code and immediately executing the unreviewed change against production.

3. What should a controller job record prove?

Who launched it, what template, revision, inventory and resolved targets, credential references, EE digest, inputs, events, result, and artifacts.

4. Why is controller backup more than a database dump?

Recovery can also depend on secret keys, configuration, certificates, projects, registry images, external credentials, and integration settings.

Chapter Summary

- The controller is a privileged change-control plane.
- RBAC, immutable run evidence, capacity, and tested recovery enforce safe separation of duties.

Chapter 35: Job Templates, Surveys, Workflows, Schedules, Notifications, and API Integration

Enterprise users consume automation through job templates and workflows rather than raw command lines. A well-designed service constrains inputs, selects trusted content and credentials, includes approvals and validation, exposes useful status, and prevents users from expanding scope beyond their authorization.

Learning Objectives

- Design safe job templates with constrained prompts and surveys.
- Build workflow job templates with success, failure, always, and approval paths.
- Use schedules, notifications, callbacks, and the REST API responsibly.
- Model self-service automation as a governed product with SLOs and support ownership.

Core Concepts

Job template: Binds a project playbook, inventory, credentials, EE, instance group, limits, tags, verbosity, and prompt policy into a repeatable launch definition.

Survey: Collects typed user input. Survey validation improves usability but server-side playbook assertions must still enforce security and domain constraints.

Workflow template: Connects jobs, project or inventory updates, approvals, and other workflows through success, failure, or always edges.

Approval node: Introduces human authorization with timeout and audit evidence. It should identify reviewer role, decision criteria, and escalation path.

Schedule: Runs a template at a defined time. Schedules need timezone, maintenance-window, ownership, overlap, failure-notification, and disablement controls.

API integration: The REST API enables service portals and pipelines. Tokens must be short-lived or protected, scoped, rotated, and associated with a nonhuman identity and clear audit trail.

Feature	Safe Use	Risk
Prompt on launch	Allow only bounded, typed fields	User changes inventory or credential
Survey	Business input with validation	Trusting UI validation alone
Workflow	Explicit gates and failure routes	Complex graph hides partial success
Schedule	Approved recurring maintenance	Overlap or forgotten automation
Notification	Actionable result and link	Secret or excessive event leakage

Reference Configuration or Command Pattern

```
# Workflow design (conceptual)
Sync Project
-> Validate Inventory
-> Syntax and Policy Check
-> Canary Deployment
-> Functional Validation
-> Approval: Production Rollout
-> Serial Production Deployment
-> Post-Change Validation

Failure edges:
Canary Deployment -> Recovery Workflow
Production Deployment -> Incident Notification
Always edge:
Any terminal node -> Evidence and Ticket Update

# REST launch pattern (illustrative)
POST /api/v2/job_templates/42/launch/
{
  "limit": "canary",
  "extra_vars": {
    "application_version": "2026.07.31"
  }
}
```

Production Checklist

- Templates do not permit users to prompt for unrestricted inventory, credentials, SCM branch, or arbitrary extra variables.
- Survey fields have type, choices, bounds, help text, defaults, and sensitive handling.
- Playbooks repeat authorization and data validation rather than trusting portal inputs.
- Workflow failure paths preserve partial-state evidence and route to recovery or incident handling.
- Schedules and API tokens have owners, expiration or review dates, and alerting for failure or misuse.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Survey value bypasses expected constraint	Validation exists only in UI or variable can be supplied elsewhere	Add playbook assertions and controller prompt restrictions	Invalid value fails before change
Workflow reports success despite failed optional node	Edge semantics or convergence setting misunderstood	Review node status rules and test every path	Terminal result matches policy
Scheduled job overlaps previous run	Long execution and no serialization control	Use workflow lock, maintenance window, or external coordination	Only one active change per target domain
API token launches broad scope	Service identity has excessive roles	Scope identity and template; lock limits and prompts	Only the intended launch is permitted

Hands-on Lab 35: Build a Governed Self-Service Deployment Workflow

Prerequisites

- AWX/controller lab or a controller-resource design simulation.
- A validation playbook, canary playbook, rollout playbook, and health test.
- Developer, operator, approver, and service-account identities.

Implementation

1. Create a job template exposing only application version and approved environment choices.
2. Add playbook assertions for version format, environment, and target limits.
3. Build a workflow with project sync, validation, canary, health check, approval, rollout, post-validation, and evidence nodes.
4. Create failure edges to recovery and notification templates.
5. Test success, validation failure, canary failure, approval rejection, production failure, and notification failure paths.
6. Launch through the API with a scoped token; record the audit chain, SLOs, owner, maintenance window, and deprecation process.

Validation Evidence

- Every workflow path terminates with an accurate status and evidence.
- The API identity cannot alter protected inventory, credentials, branch, or EE.
- Approval records reviewer and decision.
- The service documentation identifies inputs, outputs, risks, rollback, SLO, and support process.

Cleanup

- Revoke the API token and disable schedules.
- Delete temporary notifications and credentials.
- Retain sanitized workflow diagram, RBAC tests, and job event evidence.

Interview Questions and Model Answers

1. What does a job template provide?

A controlled launch contract that binds trusted content, target scope, credentials, execution environment, prompts, and execution settings.

2. Why validate survey input again in the playbook?

Inputs can arrive through APIs, workflows, or future configuration changes; server-side validation protects the automation boundary.

3. How should workflow failures be designed?

Preserve the original failure, identify partial changes, route to bounded recovery or incident response, and always capture evidence.

4. What is the main self-service automation risk?

A friendly portal can conceal excessive permissions or unbounded inputs, allowing users to execute privileged changes outside intended scope.

Chapter Summary

- Templates and surveys are governed interfaces, not substitutes for server-side authorization and validation.
- Workflow paths, self-service constraints, ownership, SLOs, and lifecycle must be tested and auditable.

Chapter 36: Windows Automation with WinRM, PSRP, SSH, PowerShell, and DSC

Windows managed nodes differ from POSIX systems in connection protocols, shells, modules, path semantics, privilege, reboots, update behavior, and desired-state facilities. Reliable Windows automation uses Windows-specific collections and transports rather than translating Linux tasks mechanically.

Learning Objectives

- Configure secure Windows connectivity through PSRP, WinRM, or SSH.
- Use `ansible.windows` and `community.windows` modules correctly.
- Manage packages, services, registry, files, users, updates, and reboots idempotently.
- Integrate PowerShell and Desired State Configuration without losing change control.

Core Concepts

Connection transport: PSRP and WinRM use PowerShell remoting; SSH is also supported in appropriate Windows configurations. Authentication, encryption, certificates, proxies, and domain trust determine the safe choice.

Windows modules: `ansible.windows` and `community.windows` provide platform-aware modules for services, features, registry, files, packages, updates, users, and reboots.

Privilege: Windows elevation, token filtering, UAC, local versus domain accounts, and become methods differ from `sudo`. Test the exact security context.

Reboots: Many changes require restart coordination. `win_reboot` waits for shutdown, startup, connectivity, and test-command readiness.

PowerShell: `win_powershell` returns structured data and supports check-mode conventions when scripts are designed accordingly. Raw shell use requires explicit change and error semantics.

DSC: `win_dsc` can invoke available Desired State Configuration resources. DSC resource versions and PowerShell dependencies must be controlled in the EE and target.

Area	Preferred Mechanism	Evidence
Service	<code>ansible.windows.win_service</code>	State, start mode, second run
Registry	<code>ansible.windows.win_regedit</code>	Key/value verification
Feature	<code>ansible.windows.win_feature</code>	Feature result and reboot flag
Update	<code>ansible.windows.win_updates</code>	Classification, reboot, post-check
Script	<code>ansible.windows.win_powershell</code>	Structured output and change rule

Reference Configuration or Command Pattern

```
# inventory/group_vars/windows.yml
ansible_connection: psrp
ansible_port: 5986
ansible_psrp_protocol: https
ansible_psrp_auth: negotiate
ansible_psrp_cert_validation: validate

# playbook excerpt
- name: Manage Windows service
  ansible.windows.win_service:
    name: W32Time
    start_mode: auto
    state: started

- name: Set application registry value
  ansible.windows.win_regedit:
    path: HKLM:\Software\Acme\Example
    name: Environment
    data: production
    type: string

- name: Reboot only when required
  ansible.windows.win_reboot:
    reboot_timeout: 1800
  when: feature_result.reboot_required | default(false)
```

Production Checklist

- Remoting uses TLS and certificate validation where feasible; plaintext transport is prohibited.
- Credentials use least privilege, protected controller credential types, and domain-aware authentication.
- Windows modules are preferred over generic command execution.
- Reboots are serialized, health-checked, and coordinated with service dependencies.
- PowerShell and DSC dependencies are versioned, tested, and documented.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
UNREACHABLE over WinRM/PSRP	Listener, TLS, firewall, authentication, proxy, or certificate issue	Test remoting independently and inspect exact transport settings	Secure remote command succeeds and Ansible ping works
Access denied despite administrator account	UAC token filtering, logon rights, delegation, or local-account restriction	Inspect security token and authentication model; use approved domain or service identity	Task runs with intended effective privileges
win_updates never completes	Reboot loop, update service issue, proxy, or long scan	Use categories, logs, timeout, serial canary, and maintenance window	Updates converge and post-reboot health passes
PowerShell task always changes	Script lacks explicit state comparison	Return structured before/after state and set change	Second run is unchanged

Hands-on Lab 36: Automate a Windows Baseline Safely

Prerequisites

- A disposable Windows Server VM or approved sandbox.
- Secure PSRP, WinRM, or SSH connectivity.
- A Windows-capable Execution Environment with required collections.

Implementation

1. Validate secure connectivity and gather a minimal Windows fact set.
2. Create a role that manages a feature, directory, configuration file, registry value, local group, and service.
3. Add a PowerShell task that returns structured validation data and correct changed status.
4. Introduce a controlled reboot condition and test serial execution.
5. Run supported check mode, apply, then perform a second idempotency run.
6. Test invalid certificate, insufficient privilege, and failed health scenarios; document authentication, reboot, rollback, and emergency access.

Validation Evidence

- Transport encryption and certificate validation are proven.
- All configuration is verified through Windows-native queries.
- The second run reports no unnecessary changes.
- Reboot and failure scenarios preserve service availability and evidence.

Cleanup

- Restore the VM snapshot or remove lab configuration.
- Revoke temporary accounts and certificates.
- Retain sanitized role, transport matrix, and test evidence.

Interview Questions and Model Answers

1. Why can Windows not be treated like Linux in Ansible?

It uses different transports, shells, privilege models, modules, path and registry semantics, update behavior, and reboot requirements.

2. When should win_powershell be used?

For Windows logic not covered by a desired-state module, with structured output, explicit change and failure semantics, and safe input handling.

3. What is the value of PSRP?

It uses the PowerShell Remoting Protocol and provides a platform-native remote execution path with supported authentication and encryption options.

4. How should Windows reboots be automated?

Only when required, in controlled batches, with timeout, connectivity recovery, application health checks, and dependency-aware rollback or incident handling.

Chapter Summary

- Use Windows-specific modules, secure transports, and proven effective privilege.
- Treat reboots as controlled workflow events with recovery and health validation.
- PowerShell and DSC dependencies and change semantics must be versioned and tested.

Chapter 37: Network Automation, Persistent Connections, Resource Modules, Backups, and Validation

Network devices often cannot run Python modules and expose configuration through CLI, NETCONF, or HTTP APIs. Network automation therefore executes primarily on the control node and depends on persistent connection plugins, platform collections, structured resource modules, configuration backups, diff review, and safeguards against management-plane loss.

Learning Objectives

- Explain why network module execution differs from server automation.
- Configure network_cli, netconf, and httpapi persistent connections.
- Use network resource-module states safely across platforms.
- Back up, render, parse, validate, deploy, and verify network configuration.

Core Concepts

Control-node execution: Network modules generally run on the control node because many devices cannot execute Python. Controller CPU, dependencies, and connection persistence therefore matter.

Persistent connection: network_cli, netconf, and httpapi maintain device sessions. ansible_network_os selects platform behavior and collection plugins.

Resource modules: Platform resource modules manage structured domains such as interfaces, VLANs, routes, or ACLs using states such as gathered, rendered, parsed, merged, replaced, overridden, and deleted.

State semantics: merged adds or updates provided settings; replaced replaces a resource subsection; overridden can remove all unspecified resources and is high risk.

Offline validation: rendered and parsed states allow data models and native configurations to be checked without changing a device.

Change safety: Back up running state, limit to canary devices, protect management interfaces, validate reachability, use maintenance windows, and test rollback or replace-confirm behavior where the platform supports it.

State	Meaning	Risk
gathered	Read structured resource state	Read permissions and scale
rendered	Generate native commands offline	Platform version mismatch
parsed	Convert native config to data	Parser coverage and normalization
merged	Add/update supplied configuration	Residual unwanted settings
overridden	Make supplied list authoritative	Loss of access or unrelated config

Reference Configuration or Command Pattern

```
# group_vars/ios.yml
ansible_connection: ansible.netcommon.network_cli
ansible_network_os: cisco.ios.ios
ansible_become: true
ansible_become_method: enable

- name: Gather interface resource state
  cisco.ios.ios_interfaces:
    state: gathered
  register: current_interfaces

- name: Render proposed interface configuration offline
  cisco.ios.ios_interfaces:
    config: "{{ desired_interfaces }}"
    state: rendered
  register: rendered_interfaces

- name: Merge approved interface settings
  cisco.ios.ios_interfaces:
    config: "{{ desired_interfaces }}"
    state: merged
```

Production Checklist

- Each device group defines the correct connection plugin and ansible_network_os FQCN.
- Credentials and enable secrets are injected through approved credential systems.
- The running configuration or relevant resource state is backed up before mutation.
- Management interfaces, routing, authentication, and controller reachability have explicit protection rules.
- Every change includes offline render or parse review, canary application, post-change validation, and rollback criteria.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Unable to open shell	Connection plugin, network_os, prompt, privilege, or timeout issue	Enable network debugging carefully; verify manual protocol and platform settings	Persistent session and simple show command succeed
Config module always changed	Abbreviated commands, normalization, or unsupported idempotency	Use resource module or full canonical commands; compare gathered state	Second run is unchanged
Device access lost after change	Management interface, route, ACL, or auth modified	Use out-of-band access and approved rollback; preserve evidence	Controller and operator connectivity restored
Backup written to unexpected location	Network backups are created on control node or EE filesystem	Set and mount a controlled artifact path with retention rules	Backup is available, protected, and linked to job

Hands-on Lab 37: Build a Safe Multi-Vendor Network Change Pipeline

Prerequisites

- Vendor sandbox, virtual network lab, or offline rendered configuration only.
- Approved network collections in the EE.
- Out-of-band or snapshot recovery for any mutable device.

Implementation

1. Create inventory groups by network platform and define persistent connection settings.
2. Gather facts and selected resource state without changing devices.
3. Model a small interface or VLAN change as structured data.
4. Use rendered state offline and review the native commands.
5. Back up current configuration and apply to one canary device with merged state.
6. Validate reachability, routing, interface state, and the exact resource configuration.
7. Run again for idempotency; test a rejected unsafe overridden change in validation only.
8. Document device backup, rollback, maintenance, timeout, and access-loss procedures.

Validation Evidence

- Offline render matches approved intent.
- Canary change preserves management connectivity.
- Post-change gathered state matches desired structured data.
- Second run reports no unnecessary change and backup evidence is retained securely.

Cleanup

- Restore the original lab configuration or reset the virtual device.
- Remove local backups according to evidence-retention policy.
- Revoke temporary credentials and close lab access.

Interview Questions and Model Answers

1. Why do network modules usually execute on the control node?

Network devices often cannot run Python modules, so the control node communicates through CLI, NETCONF, or HTTP APIs.

2. What is dangerous about overridden state?

It can remove all resource entries not present in the supplied data, including management-critical configuration.

3. Why use rendered or parsed states?

They support offline review and data-model validation without changing a device.

4. How should network rollback be designed?

Use backups, vendor commit-confirm or checkpoint features where available, out-of-band access, canaries, explicit timeout criteria, and tested restoration commands.

Chapter Summary

- Network automation uses platform connections and structured resource models.
- Persistent connection configuration is part of inventory architecture.
- Offline render, backup, canary, validation, and rollback protect the management plane.
- Resource-module state choice is a high-consequence design decision.

Chapter 38: ansible-lint, Molecule, pytest-ansible, CI, and Release Qualification

Automation tests must prove more than YAML syntax. A production release should verify style and risky patterns, role interfaces, supported platforms, convergence, idempotency, handlers, check mode, failure recovery, security controls, and compatibility with the intended ansible-core, collections, Python, and execution environment.

Learning Objectives

- Build a layered test strategy for roles and collections.
- Use ansible-lint and yamllint as enforceable quality gates.
- Create Molecule scenarios and pytest-based assertions.
- Design CI matrices and release qualification for supported platforms and dependencies.

Core Concepts

Static checks: YAML parsing, ansible-lint, FQCN use, schema validation, secret scanning, and policy checks catch defects before execution.

Molecule scenario: Defines dependency, create, prepare, converge, idempotence, side_effect, verify, and destroy stages against disposable instances.

pytest-ansible: Integrates Ansible execution and Molecule scenarios with pytest, enabling test fixtures and structured assertions.

Test matrix: Covers minimum and maximum supported ansible-core, collection dependencies, Python versions, target OS families, and execution environments.

Negative testing: Deliberately invalid inputs, missing dependencies, failed health checks, revoked credentials, interrupted runs, and recovery paths must be tested.

Release gate: A release candidate is built once, installed into a clean EE, tested from the artifact, scanned, and promoted only when evidence is complete.

Layer	Typical Tool	Question
Syntax/schema	ansible-playbook, schema checks	Can content be parsed?
Style/risk	ansible-lint, yamllint	Does it follow safe conventions?
Convergence	Molecule	Does it reach desired state?
Idempotency	Molecule idempotence	Does a second run stay stable?
Acceptance	pytest/health probes	Does the service actually work?

Reference Configuration or Command Pattern

```
# .config/ansible-lint.yml
profile: production
skip_list: []
warn_list:
  - experimental

# molecule/default/converge.yml
- name: Converge
  hosts: all
  become: true
  roles:
    - role: acme.operations.web_service

# molecule/default/verify.yml
- name: Verify
  hosts: all
  tasks:
    - name: Check service health
      ansible.builtin.uri:
        url: http://127.0.0.1:8080/health
        status_code: 200

# Typical sequence
ansible-lint
molecule test
pytest -q
```

Production Checklist

- Lint configuration is version controlled and rule exceptions include owner, reason, scope, and expiration.
- Molecule tests the installed collection artifact or release candidate, not only the source tree.
- Tests include first convergence, idempotency, functional validation, and cleanup.
- CI matrices cover supported versions without using mutable latest tags.
- Secrets, controller tokens, images, and test artifacts are protected and automatically removed.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Molecule passes locally but fails in CI	Different container runtime, image, dependency, architecture, or privilege	Run the same EE and pinned scenario; compare environment details	Local and CI use equivalent immutable inputs
Idempotence test changes every run	Volatile template, command, package metadata, or handler loop	Inspect diff and changed tasks; make desired state deterministic	Second converge reports changed=0
ansible-lint rule disabled globally	Legacy defect or rushed exception	Narrow exclusion and create remediation deadline	Rule enabled for new content and exception tracked
Verify passes after failed converge	Stale scenario state or bad ordering	Destroy first, fail fast, and assert converge	Clean failure is detected

Hands-on Lab 38: Create a Release-Quality Test Pipeline for a Collection

Prerequisites

- The collection from Chapter 32.
- A container runtime and compatible Molecule driver.
- CI system or a documented local matrix simulation.

Implementation

1. Configure yamllint, ansible-lint production profile, secret scanning, and collection sanity checks.
2. Create a Molecule scenario with prepare, converge, verify, idempotence, negative side effect, and destroy stages.
3. Add pytest assertions for rendered configuration and service response.
4. Test minimum and current supported ansible-core and collection dependencies in separate EEs.
5. Build the collection artifact, rerun from the installed tarball, and publish versions, image digests, results, limitations, and artifacts.

Validation Evidence

- Static, convergence, idempotency, negative, and acceptance tests pass.
- The suite fails when a deliberate unsafe task or bad interface is introduced.
- The tested commit, release artifact, immutable environment, and cleanup evidence are traceable.

Cleanup

- Remove test containers, networks, volumes, and credentials.
- Delete logs containing sensitive environment details according to policy.
- Retain sanitized test configuration and release report.

Interview Questions and Model Answers

1. What does ansible-lint add beyond yamllint?

It understands Ansible task semantics and safe-content conventions, while yamllint primarily checks YAML structure and style.

2. What should an Ansible idempotency test prove?

A second convergence run with unchanged inputs causes no unnecessary changes while the functional state remains correct.

3. Why test the built collection artifact?

Packaging can omit files, alter metadata, or resolve dependencies differently from the source checkout.

4. What belongs in a compatibility matrix?

Supported ansible-core, Python, collection dependency, target platform, execution environment, and controller combinations.

Chapter Summary

- Test automation behavior, not only syntax.
- Idempotency and functional acceptance are separate checks.
- Qualify immutable release artifacts and document evidence-backed compatibility and exceptions.

Chapter 39: Strategies, Forks, Serial, Async, Throttling, Performance, and Large-Fleet Safety

Ansible can execute across many hosts, but unplanned concurrency can overload the control plane, targets, load balancers, package repositories, identity systems, databases, or external APIs. Large-fleet engineering balances throughput with bounded failure, fairness, observability, and dependency capacity.

Learning Objectives

- Explain linear, free, and host-pinned execution behavior.
- Tune forks, serial, throttle, async, poll, and task timeouts.
- Design canary and rolling strategies with failure budgets.
- Measure and troubleshoot performance across controller, network, target, and dependency layers.

Core Concepts

Strategy plugin: linear keeps hosts aligned by task batches; free allows each host to progress independently; host_pinned combines host-oriented execution with worker limits.

Forks: Controls the number of worker processes. More forks can improve throughput but increase memory, CPU, sockets, target load, and external dependency pressure.

Serial: Splits the host set into batches. Percentages, lists, max_fail_percentage, and any_errors_fatal shape rollout and failure containment.

Throttle: Limits concurrency for a particular task, block, or play even when more forks are available.

Async and poll: Allows long-running operations to continue while the controller checks later. Fire-and-forget requires a separate confirmation and recovery design.

Performance evidence: Measure queue time, setup/fact time, connection establishment, task duration, retries, changed count, target load, API rates, controller CPU/memory, and artifact volume.

Control	Primary Effect	Use Case
forks	Global worker concurrency	Controller throughput
serial	Host rollout batch	Failure containment
throttle	Task-specific concurrency	Shared database or API protection
async/poll	Long operation scheduling	Patching or image build
strategy	Host/task progression	Dependency and ordering model

Reference Configuration or Command Pattern

```
- name: Rolling application update
hosts: application
serial:
  - 1
  - 10%
  - 25%
max_fail_percentage: 10
strategy: linear

tasks:
  - name: Drain node from load balancer
    ansible.builtin.uri:
      url: "https://lb.example.invalid/nodes/{{ inventory_hostname }}/drain"
      method: POST
    delegate_to: localhost
    throttle: 2

  - name: Start long upgrade asynchronously
    ansible.builtin.command: /usr/local/bin/upgrade-app
    async: 1800
    poll: 0
    register: upgrade_job

  - name: Wait for upgrade completion
    ansible.builtin.async_status:
      jid: "{{ upgrade_job.ansible_job_id }}"
    register: upgrade_status
    until: upgrade_status.finished
    retries: 120
    delay: 10
```

Production Checklist

- Concurrency is derived from measured controller and dependency capacity, not copied from another environment.
- Production rollouts begin with a canary and expand only after functional validation.
- Shared dependencies have task-level throttle and rate-limit handling.
- Async work has durable job IDs, timeout, status, cancellation, and recovery logic.
- Failure thresholds, abort behavior, and partial-fleet state are documented and rehearsed.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Run is slower with more forks	Controller contention, target throttling, connection setup, or API rate limit	Profile task and resource metrics; reduce concurrency or use persistence	Throughput improves without higher failure rate
Free strategy violates dependency order	Hosts progress independently past shared gate	Use linear or explicit coordination for global dependencies	Order-sensitive test passes

Symptom	Likely Cause	Response	Validation
Async job lost	Artifact directory removed, host rebooted, or job ID not retained	Persist job metadata and design external confirmation/recovery	Every started job reaches known terminal state
Serial rollout leaves mixed versions	Failure threshold stopped after partial batches	Trigger recovery or compatibility mode and record affected hosts	Fleet returns to approved homogeneous or supported mixed state

Hands-on Lab 39: Benchmark and Safely Scale a Rolling Automation Job

Prerequisites

- At least five disposable targets or simulated inventory hosts.
- A harmless task with measurable duration.
- Controller or local system metrics.

Implementation

1. Run a baseline with forks=2, linear strategy, and serial=1; record timing and resource metrics.
2. Repeat with controlled fork increases while keeping target work identical.
3. Introduce throttle on a simulated shared dependency and measure effect.
4. Create a canary-to-percentage serial rollout with health gates.
5. Add a long-running async task with durable status collection.
6. Inject one target failure; document thresholds, partial state, capacity envelopes, and defaults for small, medium, and large jobs.

Validation Evidence

- Results identify the bottleneck while controller and targets remain within safe limits.
- Failure stays inside the intended batch and recovery is explicit.
- Every asynchronous operation has a durable terminal result.

Cleanup

- Stop and delete simulated long-running jobs and remove temporary metrics or artifacts according to retention policy.
- Restore inventory and configuration defaults.

Interview Questions and Model Answers

1. What does forks control?

The number of Ansible worker processes available for concurrent host task execution, subject to strategy and other limits.

2. When should free strategy be avoided?

When hosts must remain synchronized at task boundaries or share dependencies that require global ordering.

3. How do serial and throttle differ?

Serial controls the host batch for the play; throttle limits concurrency for a specific task, block, or play section.

Chapter Summary

- Measure concurrency, contain rollout risk with canaries and throttles, and track every asynchronous job to a durable terminal state.

Chapter 40: Event-Driven Ansible, Rulebooks, Observability, Governance, and Enterprise Capstone

Event-driven automation connects observations to decisions and actions. It can reduce response time, but automatic remediation can also amplify false signals, repeat destructive actions, or hide underlying defects. Mature systems separate event ingestion, correlation, rule evaluation, authorization, action execution, verification, and incident evidence.

Learning Objectives

- Explain Event-Driven Ansible sources, rules, conditions, actions, and decision environments.
- Design deduplication, suppression, correlation, approvals, and circuit breakers.
- Integrate observability and controller workflows with bounded remediation.
- Complete an enterprise capstone covering content, runtime, controller, platforms, tests, scale, and events.

Core Concepts

Rulebook: YAML content defining event sources and rule conditions with actions such as `run_playbook`, `run_job_template`, `run_workflow_template`, `set_fact`, or `post_event`.

Decision environment: A containerized runtime for ansible-rulebook and its source or event-filter collection dependencies, analogous to an EE for automation execution.

Event source: A plugin that receives or polls events from a message bus, webhook, monitoring system, file, URL, or custom integration.

Correlation: Rules should reason over identity, time window, sequence, count, topology, maintenance state, and prior actions rather than reacting to one raw alert.

Action boundary: Low-risk reversible diagnostics may run automatically; high-impact mutation often requires approval or a controller workflow with guardrails.

Closed-loop verification: After action, verify the symptom, service health, user impact, and root condition. If verification fails, stop automation and escalate with evidence.

Layer	Question	Required Control
Source	Can the event be trusted?	Authentication, schema, replay protection
Decision	Is it actionable?	Correlation, suppression, maintenance context
Authorization	May automation act?	Risk class, approval, scoped identity
Action	What changes?	Template constraints, canary, circuit breaker
Verification	Did it recover?	Health test and incident evidence

Reference Configuration or Command Pattern

```
# rulebooks/service_recovery.yml
- name: Respond to repeated service health failures
  hosts: all
  sources:
    - acme.events.webhook:
        host: 0.0.0.0
        port: 5000
        token_env: EDA_WEBHOOK_TOKEN
  rules:
    - name: Collect diagnostics after three matching failures
      condition: event.payload.count >= 3 and
        event.payload.environment == "production"
      action:
        run_job_template:
          name: Collect Service Diagnostics
          organization: Platform Operations
        job_args:
          extra_vars:
            incident_id: "{{ event.payload.incident_id }}"
            service: "{{ event.payload.service }}"

    - name: Route approved recovery request
      condition: event.payload.recovery_approved == true
      action:
        run_workflow_template:
          name: Controlled Service Recovery
          organization: Platform Operations
```

Production Checklist

- Event sources authenticate senders, validate schema, constrain payload size, and prevent unauthorized replay.
- Rules deduplicate events, honor maintenance windows, and track prior actions.
- Action identities and templates are scoped to the exact service and risk class.
- Automatic mutation has rate limits, circuit breakers, maximum attempts, and escalation paths.
- Events, decisions, launches, controller job IDs, verification, and human approvals share a correlation identifier.

Troubleshooting Matrix

Symptom	Likely Cause	Response	Validation
Rule fires repeatedly for one incident	No deduplication, state tracking, or suppression window	Correlate by incident and resource; store action state; add cooldown	One bounded action sequence per incident
False alert triggers remediation	Weak source trust or single-signal rule	Require multiple signals, duration, and service-context checks	Synthetic false event produces diagnostics only

Symptom	Likely Cause	Response	Validation
Action succeeds but service remains unhealthy	Symptom not root cause or validation incomplete	Stop retries, preserve evidence, escalate to incident command	Circuit breaker opens and human owner is notified
Decision environment cannot load source	Missing collection or Python dependency	Pin dependency and test startup	Validated event is received

Hands-on Lab 40: Enterprise Ansible Automation Platform Capstone

Prerequisites

- Completed Chapters 21-39.
- Disposable Linux and Windows targets plus a network sandbox or offline network render.
- AWX/controller lab or a detailed resource simulation.
- Custom collection, EE, CI pipeline, and event source simulator.

Implementation

1. Package Linux, Windows, and network automation into a versioned internal collection with argument specs and documentation.
2. Build a signed or checksum-verified EE and decision environment from pinned dependencies; record image digests and SBOM summaries.
3. Create controller organizations, teams, projects, inventories, credentials, templates, and RBAC for developer, operator, approver, and auditor roles.
4. Build a workflow with project sync, validation, canary, approval, rollout, recovery, notifications, and evidence update.
5. Run release qualification with lint, Molecule, idempotency, Windows validation, offline network rendering, and compatibility matrices.
6. Benchmark rollout settings and define safe forks, serial, throttle, and failure thresholds.
7. Create a rulebook that accepts authenticated simulated service events, correlates repeated failures, launches diagnostics automatically, and requires approval before recovery.
8. Inject project sync failure, expired credential, Windows reboot failure, network validation failure, controller capacity exhaustion, and event storm; execute runbooks.
9. Produce architecture, threat model, dependency inventory, RBAC matrix, service catalog entry, operational SLOs, backup/restore plan, incident records, and cleanup evidence.

Validation Evidence

- Every run identifies immutable content revision, collection version, EE or decision-environment digest, inventory and targets, credential reference, inputs, approvals, results, and validation.
- Role, collection, controller, Windows, network, performance, and event tests pass from clean environments.
- Fault injections remain within the defined blast radius and follow documented recovery.
- No credential, production address, private artifact, or secret appears in source, images, logs, or portfolio evidence.

Cleanup

- Destroy disposable targets and controller lab resources.
- Revoke service tokens, certificates, test credentials, and registry access.
- Remove images and artifacts not retained under the approved evidence policy.
- Retain a sanitized portfolio repository, diagrams, runbooks, test report, and lessons learned.

Interview Questions and Model Answers

1. What is Event-Driven Ansible?

A capability that evaluates events through rulebooks and can invoke playbooks or controller templates when conditions match.

2. What is a decision environment?

A container image containing ansible-rulebook and the collections and dependencies required for event sources, filters, and rule processing.

3. Which remediations should be fully automatic?

Only low-risk, reversible, well-tested actions with strong signals, scoped identity, rate limits, circuit breakers, and closed-loop verification.

4. What demonstrates enterprise Ansible maturity?

Versioned content and runtimes, controlled controller resources, least-privilege credentials, tested multi-platform automation, release evidence, bounded scaling, event safeguards, recovery drills, and reproducible audit trails.

Chapter Summary

- Event-driven automation must correlate, authorize, bound, and verify actions.
- Decision environments are governed runtime artifacts.
- The enterprise capstone unifies content engineering, execution images, controller operations, platform automation, testing, scale, and event response.
- Evidence and recovery are first-class automation outputs.

Part D Advanced Ansible Command Reference

Run all commands from a controlled repository and pinned environment. Review inventory, credentials, image digest, target limit, and artifact paths before execution.

Purpose	Command	Operational Note
Initialize role	<code>ansible-galaxy role init roles/NAME</code>	Add argument specs, tests, README, and owner before reuse.
Initialize collection	<code>ansible-galaxy collection init NAMESPACE.NAME</code>	Review namespace, license, runtime metadata, and release process.
Build collection	<code>ansible-galaxy collection build</code>	Inspect tarball manifest and produce in CI.
Install collection	<code>ansible-galaxy collection install ARTIFACT -p PATH</code>	Use project-local path for clean qualification.
List collections	<code>ansible-galaxy collection list</code>	Capture exact versions used by a run.
Build EE	<code>ansible-builder build --tag IMAGE</code>	Record builder version, base digest, SBOM, scan, and result digest.
Inspect EE	<code>ansible-navigator images --mode stdout</code>	Verify core, Python, collections, user, and dependencies.
Run in EE	<code>ansible-navigator run PLAYBOOK -i INVENTORY --mode stdout</code>	Pin or record the image digest.
Runner invocation	<code>ansible-runner run PRIVATE_DATA_DIR -p PLAYBOOK</code>	Protect artifacts and use absolute inventory paths.
Lint	<code>ansible-lint</code>	Use version-controlled profile and tracked exceptions.
Molecule full test	<code>molecule test</code>	Creates, converges, verifies, checks idempotency, and destroys.
Molecule converge	<code>molecule converge</code>	Useful during role development; do not replace full release test.
Molecule verify	<code>molecule verify</code>	Runs acceptance verification against scenario instances.
Windows ping	<code>ansible windows -m ansible.windows.win_ping</code>	Validates Windows remoting and module execution.
Network command	<code>ansible routers -m ansible.netcommon.cli_command -a "command=show version"</code>	Use platform inventory and protected credentials.
Rulebook run	<code>ansible-rulebook --rulebook rulebooks/main.yml -i inventory</code>	Use a pinned decision environment in production.

Purpose	Command	Operational Note
AWX API browse	GET /api/v2/	Use scoped tokens and TLS; do not expose tokens in shell history.
Controller launch	POST /api/v2/job_templates/ID/launch/	Template must constrain prompts, inventory, credentials, and EE.

Advanced Ansible Production Readiness Checklist

Content Packaging

- ☐ Roles have narrow ownership, argument specifications, documentation, tests, and changelogs.
- ☐ Collections use FQCNs, semantic releases, controlled dependencies, and immutable artifacts.
- ☐ Deprecations and breaking changes include migration guidance and tested redirects where appropriate.

Runtime Supply Chain

- ☐ Execution and decision environments use approved base images and pinned dependencies.
- ☐ Images have SBOM, vulnerability scan, provenance or signature, and immutable digest.
- ☐ No credential, inventory secret, or environment-specific private data is stored in image layers.

Controller Operations

- ☐ Organizations, teams, resource roles, and credentials enforce separation of duties.
- ☐ Projects use protected SCM revisions and jobs record the exact commit.
- ☐ Controller, database, secret keys, certificates, registry, and integrations have tested recovery.

Multi-Platform Safety

- ☐ Windows transport, authentication, certificates, privilege, reboots, and health checks are tested.
- ☐ Network connection plugins, platform collections, backups, canaries, and management-plane safeguards are defined.
- ☐ Platform-specific modules are used instead of unsafe generic commands.

Testing and Scale

- ☐ Lint, schema, secret, convergence, idempotency, negative, and acceptance tests are required.
- ☐ Supported ansible-core, collection, Python, target platform, EE, and controller combinations are tested.
- ☐ Forks, serial, throttle, async, failure thresholds, and capacity envelopes are documented.

Events and Evidence

- ☐ Event sources are authenticated, schema-validated, deduplicated, and replay-protected.
- ☐ Automatic actions have risk classification, scoped identity, rate limit, circuit breaker, and verification.
- ☐ Correlation IDs link events, decisions, approvals, controller jobs, results, and incidents.

Ten Additional Advanced Ansible Portfolio Projects

1. Internal Role Product

Create a role with argument specs, safe defaults, handlers, tests, changelog, release process, and support runbook.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

2. Enterprise Collection Release

Package roles and plugins, test minimum/maximum dependencies, build immutable artifacts, and publish a signed release.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

3. Execution Environment Factory

Generate minimal EEs for Linux, Windows, and network automation with SBOMs, scans, and promotion workflows.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

4. Controller RBAC Blueprint

Model organizations, teams, credentials, inventories, projects, templates, approvals, and audit access for multiple departments.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

5. Self-Service Operations Portal

Integrate a small portal with scoped controller API tokens and constrained templates for safe operator actions.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

6. Windows Patch Orchestrator

Implement canary patching, reboot coordination, application validation, failure isolation, and compliance evidence.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

7. Network Configuration Compliance

Gather resource state, parse or render offline, compare policy, back up, remediate a canary, and verify idempotency.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

8. Collection Compatibility Laboratory

Test multiple ansible-core, Python, dependency, OS, and EE combinations and publish a support matrix.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

9. Large-Fleet Capacity Study

Benchmark forks, strategies, serial, throttle, connection persistence, and external dependency limits.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

10. Event-Driven Diagnostic System

Correlate monitoring events, suppress duplicates, launch diagnostics, verify outcomes, and route high-risk recovery for approval.

- Deliverables: sanitized source, immutable artifact references, architecture and threat model, tests, RBAC or permission matrix, runbooks, evidence, cleanup proof, risks, and lessons learned.

Senior Advanced Ansible Interview Drill Bank

1. How do roles and collections differ?

A role packages reusable playbook content in a known structure. A collection is a namespaced distribution unit that can include roles, modules, plugins, playbooks, documentation, metadata, and dependencies.

2. What makes an Execution Environment reproducible?

An approved immutable base, explicit ansible-core, runner, Python, collection and system dependencies, deterministic build process, image digest, and qualification evidence.

3. How does controller RBAC protect production?

It separates source editing, inventory administration, credential administration, template design, approval, launch, and audit access at object and organization boundaries.

4. Why are job surveys not a security boundary?

Inputs can arrive through other launch paths, and UI validation can change. Playbooks and templates must enforce authorization, types, bounds, and target restrictions.

5. How is Windows automation operationally different?

It uses Windows-specific transports and modules, different privilege and path semantics, PowerShell, registry and features, and coordinated reboot and update workflows.

6. Why do network modules execute on the control node?

Many network devices cannot run Python; Ansible communicates using persistent CLI, NETCONF, or HTTP API connections through platform collections.

7. What does Molecule idempotency prove?

That repeating convergence with unchanged inputs produces no unnecessary changes in the tested scenario; it does not alone prove service correctness.

8. How should Ansible concurrency be tuned?

Measure controller, target and dependency capacity, then combine forks, strategy, serial, throttle, timeouts, and async with failure containment and health gates.

9. What is a decision environment?

A container image containing ansible-rulebook and the source and filter collection dependencies used to evaluate event-driven rulebooks.

10. When should remediation require approval?

When the action is high-impact, irreversible, weakly signaled, affects broad scope, changes security or data, or lacks reliable closed-loop verification and rollback.

Glossary

Term	Definition
Ansible Builder	A tool that creates containerized Execution Environments from a definition and dependency inputs.
Ansible Navigator	A command-line and text interface for running automation in EEs and inspecting images, collections, and documentation.
Ansible Runner	A stable execution interface and artifact model used by AWX and application integrations.
Argument specification	Machine-readable role entry-point metadata that validates and documents role inputs.
AWX	The upstream web UI, REST API, and task engine for centralized Ansible automation.
Collection	A namespaced distribution format for Ansible roles, modules, plugins, playbooks, docs, and tests.
Decision environment	A container runtime for ansible-rulebook and its event-source and filter dependencies.
Execution environment	A container image that acts as an Ansible control node with pinned runtime dependencies.
FQCN	Fully Qualified Collection Name, such as <code>ansible.windows.win_service</code> .
Instance group	A controller scheduling boundary that selects eligible execution capacity for jobs.
Job template	A controller launch definition binding playbook, inventory, credentials, EE, prompts, and execution settings.
Molecule	A testing framework for Ansible roles and collections using disposable scenarios.
Network resource module	A module that manages a structured network configuration domain through defined states.
PSRP	PowerShell Remoting Protocol, a Windows remote-management transport supported by Ansible.
Role	Reusable Ansible content organized into tasks, handlers, defaults, vars, files, templates, metadata, and tests.
Rulebook	Event-Driven Ansible YAML that defines sources, conditions, and actions.
Survey	Controller-defined typed launch questions that populate job variables.

Term	Definition
Workflow job template	A graph of controller jobs, approvals, syncs, and nested workflows connected by result conditions.
WinRM	Windows Remote Management, an implementation of WS-Management used for remote administration.
Worker fork	An Ansible process used for concurrent host task execution.

Official Ansible Documentation Index

Reference	Official URL
Ansible latest documentation	https://docs.ansible.com/projects/ansible/latest/
Reusing Ansible artifacts and roles	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_reuse.html
Using Ansible collections	https://docs.ansible.com/projects/ansible/latest/collections_guide/
Collection index	https://docs.ansible.com/projects/ansible/latest/collections/index.html
Execution Environments getting started	https://docs.ansible.com/projects/ansible/latest/getting_started_ee/
Ansible Builder documentation	https://docs.ansible.com/projects/builder/en/latest/
Execution environment definition	https://docs.ansible.com/projects/builder/en/latest/definition/
Ansible Runner documentation	https://ansible.readthedocs.io/projects/runner/en/stable/
AWX documentation	https://ansible.readthedocs.io/projects/awx/en/latest/
AWX repository	https://github.com/ansible/awx
Windows guide	https://docs.ansible.com/projects/ansible/latest/os_guide/intro_windows.html
Using Ansible and Windows	https://docs.ansible.com/projects/ansible/latest/os_guide/windows_usage.html
Ansible Windows collection	https://docs.ansible.com/projects/ansible/latest/collections/ansible/windows/
Network automation guide	https://docs.ansible.com/projects/ansible/latest/network/
Network resource modules	https://docs.ansible.com/projects/ansible/latest/network/user_guide/network_resource_modules.html

Reference	Official URL
Ansible development tools	https://docs.ansible.com/projects/dev-tools/
Ansible Lint documentation	https://ansible.readthedocs.io/projects/lint/
Molecule documentation	https://ansible.readthedocs.io/projects/molecule/
pytest-ansible documentation	https://ansible.readthedocs.io/projects/pytest-ansible/
Strategy plugins	https://docs.ansible.com/projects/ansible/latest/plugins/strategy.html
Asynchronous actions and polling	https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_async.html
Ansible Rulebook documentation	https://docs.ansible.com/projects/rulebook/en/latest/
Ansible release and maintenance	https://docs.ansible.com/projects/ansible/latest/reference_appendices/release_and_maintenance.html
Ansible porting guides	https://docs.ansible.com/projects/ansible/latest/porting_guides/porting_guides.html

Ansible, ansible-core, collections, execution environments, AWX, Automation Controller, Windows transports, network platform plugins, testing tools, and Event-Driven Ansible evolve independently. Before production release, verify current stable documentation, collection support matrices, porting guides, image requirements, and platform life cycles.

Volume 9D Completion

Part D completes advanced Ansible content engineering and enterprise automation operations. The next section integrates Terraform and Ansible into governed delivery pipelines with GitOps, security, compliance, secrets, observability, drift management, and cross-tool ownership controls.

Next part: Volume 9E - Integrated Terraform-Ansible Pipelines, GitOps, Security, Governance, Compliance, Observability, and Enterprise Delivery.